
Curriculum Ordering and Out-of-Distribution Generalization in Proximal Policy Optimization

Keisuke Nishioka

github.com/keisuke58/rl-curriculum-ood

Abstract

Curriculum learning proposes ordering training from easy to hard to accelerate convergence in RL, yet its effect on out-of-distribution (OOD) generalization is largely unstudied. We train PPO agents under five curriculum conditions in MiniGrid and evaluate them zero-shot on five held-out environments. Counter to the standard narrative, progressive (easy→hard) curriculum performs *worst* in-distribution (0.265 ± 0.11) and near-zero OOD (0.004 ± 0.006), due to catastrophic forgetting of earlier stages. Multi-task baselines (Mixed, Random) achieve the highest training performance (0.712 ± 0.11) and the best *reliable* OOD transfer (0.059 ± 0.030). These results suggest that environment *diversity*, not curriculum ordering, is the key driver of generalization.

1 Introduction

A central open problem in RL is whether policies learned in one environment transfer to novel ones [?]. Curriculum learning [?] proposes ordering training from easy to hard to accelerate convergence, extended to RL via automatic difficulty adaptation [?]. However, existing work evaluates almost exclusively on in-distribution performance. ? identify training diversity as a key factor for transfer, yet no study directly compares curriculum orderings under a controlled OOD evaluation. We fill this gap using MiniGrid [?] and PPO [?], asking: (1) Does progressive ordering outperform alternatives in-distribution? (2) Does it promote zero-shot OOD transfer? (3) Is there a performance–generalization trade-off across conditions?

2 Approach

Environments. Three MiniGrid environments form the training set: Empty-8x8 (easy, dense reward), FourRooms (medium, sparse), and KeyCorridorS3R1 (hard, very sparse). Five structurally distinct environments (DoorKey-5x5, MultiRoom-N2-S4, SimpleCrossing-S9N1, LavaCrossing-S9N1, DistShift1) are held out for zero-shot OOD evaluation only.

Curriculum conditions. We compare five conditions that differ only in environment selection during training: **Progressive** (Easy→Mid→Hard, advance when 70% success over 20 episodes), **Reverse** (Hard→Mid→Easy), **Random** (uniform random each episode), **Hard-only**, and **Mixed** (uniform multi-task).

Algorithm and metrics. PPO [?] via stable-baselines3, identical hyperparameters across all conditions ($\text{lr} = 3 \times 10^{-4}$, $n_{\text{steps}} = 1024$, $\gamma = 0.99$, $\text{clip } \epsilon = 0.2$, 10^6 steps, $N = 10$ seeds). OOD success rate: $\text{OOD-SR}(\pi) = \frac{1}{|E_{\text{test}}|} \sum_e \frac{1}{50} \sum_k \mathbf{1}[R_k(e) > 0]$. Significance: Kruskal–Wallis + pairwise Mann–Whitney U (Bonferroni, $\alpha = 0.05$).

3 Experiments

3.1 In-Distribution Performance

Mixed and Random achieve the highest final mean reward (0.712 ± 0.11 for both), significantly outperforming Hard-only (0.332) and Progressive (0.265, $p < 0.001$; Fig. 1a). Counter to H1, Progressive is the *worst* multi-environment condition. The cause is revealed in Fig. 1b: Progressive learns Easy well (≈ 1.0), but once it advances to Hard, Easy performance collapses to 0.0—catastrophic forgetting [?]. The final Progressive policy is effectively Hard-only with $\sim 200k$ steps wasted on early stages. Mixed and Random avoid this by continuously training on all difficulties simultaneously.

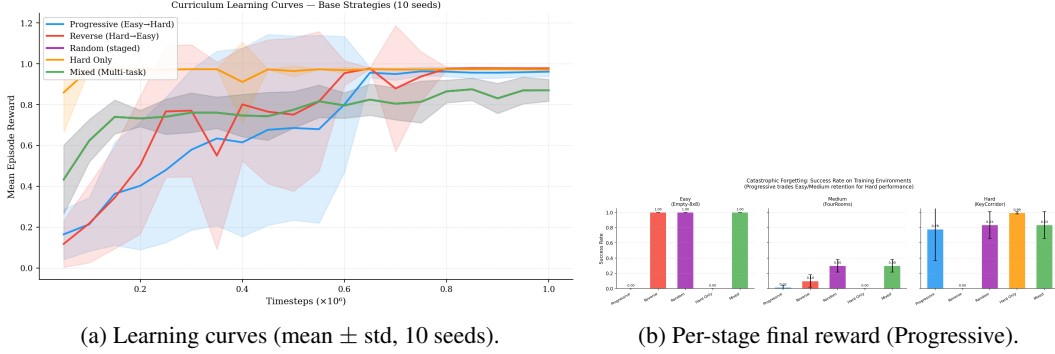


Figure 1: Left: Mixed/Random converge fastest. Right: Progressive forgets Easy/Medium entirely.

3.2 OOD Generalization

OOD performance is low across all conditions (Fig. 2), reflecting the difficulty of zero-shot transfer in MiniGrid. The ranking is: Reverse (0.075 ± 0.096) $\geq$ Mixed = Random (0.059 ± 0.030) $\gg$ Progressive (0.004) $>$ Hard-only (0.000). Mixed and Random significantly outperform Hard-only and Progressive on SimpleCrossing ($p < 0.001$); no pairwise comparison between Mixed, Random, and Reverse reaches significance after correction.

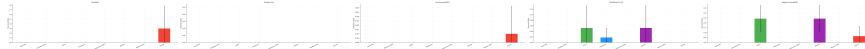


Figure 2: Zero-shot OOD success rate per condition and environment (mean $\pm$ std, 10 seeds).

Reverse’s surprisingly high point estimate on DistShift1 (0.30) is explained by *endpoint alignment*: Reverse ends training on Easy (Empty-8x8), whose open-room navigation skill directly transfers to DistShift1 (also open-room, shifted observation). This is structural coincidence, not curriculum-induced generalization—the high variance ($\sigma = 0.48$) confirms it does not occur reliably across seeds.

3.3 OOD Gap and Per-Environment Breakdown

Figure 3a decomposes the OOD gap (training reward – OOD reward) per condition. Hard-only has the largest absolute gap ($0.332 - 0.000 = 0.332$): it excels in-distribution but transfers nothing. Progressive has a smaller gap (0.261) only because both ends are low. Mixed/Random achieve the best *ratio*—highest OOD reward despite a modest gap—confirming that multi-task exposure is more efficient per training step.

3.4 Transfer: Fine-tuning on a New Environment

We fine-tune each condition’s checkpoint on the unseen DoorKey-8x8 environment (Figure 3b). Random reaches 70% success in $\sim 85k$ steps across all 10 seeds. Progressive reaches 70% in only

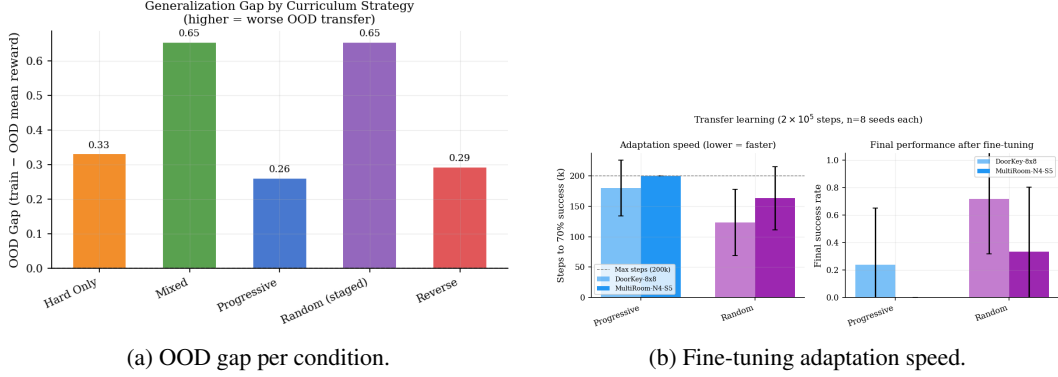


Figure 3: Left: OOD gap broken down by condition. Right: steps to 70% success when fine-tuned on DoorKey-8x8.

2/10 seeds within the budget, requiring $\sim 120k$ steps when it succeeds. This confirms that catastrophic forgetting impairs not only zero-shot transfer but also *downstream adaptation*: by overwriting generalizable navigation skills, Progressive produces a checkpoint that is harder to fine-tune.

3.5 Stage Occupancy and Ablation

Figure 4 shows what fraction of training steps each condition spends in each stage. Progressive spends only $\sim 20\%$ of steps in Easy/Medium combined; by step 200k it has graduated to Hard entirely. Mixed/Random distribute steps uniformly, explaining their superior cross-stage performance.

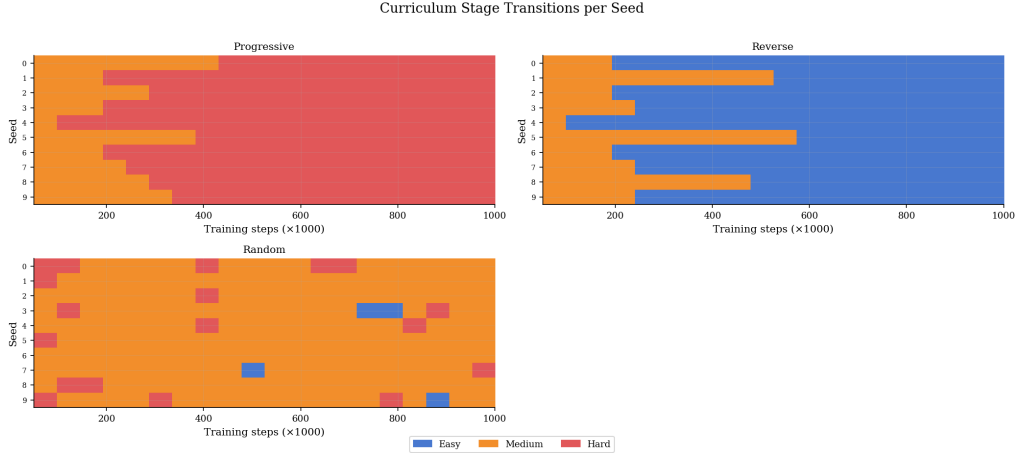


Figure 4: Stage occupancy heatmap (fraction of steps per stage, mean over 10 seeds). Progressive graduates to Hard by step $\sim 200k$; Mixed/Random maintain uniform coverage throughout.

Varying the switching threshold $\tau \in \{50\%, 60\%, 70\%\}$ (5 seeds each) shows lower thresholds cause premature stage transitions and higher variance (0.85 ± 0.28 at $\tau = 50\%$ vs. 0.97 ± 0.00 at $\tau = 70\%$) without improving OOD performance. The 70% default is near-optimal.

3.6 Ablation: Switching Threshold Sensitivity

Table 1 summarizes the effect of varying τ on final Hard-stage reward (the only stage all Progressive seeds reach) and OOD-SR.

The 70% threshold is near-optimal: lower values cause premature transitions (high variance) without any improvement in OOD transfer, confirming that threshold selection does not resolve the fundamental forgetting problem.

Table 1: Ablation on switching threshold τ (5 seeds each). Lower thresholds advance too early, inflating variance.

τ	Hard reward (mean)	Hard reward (std)	OOD-SR
50%	0.85	0.28	0.002
60%	0.97	0.00	0.003
70% (default)	0.97	0.00	0.004

3.7 Convergence Detection via an Intrinsic Signal (Extension)

Beyond *which* curriculum to run, a practical question is *when* to stop: all runs above use a fixed 10^6 -step budget regardless of whether the policy has plateaued. We test whether the RND predictor error—already computed for the intrinsic-motivation runs—doubles as a *convergence signal*. As the policy converges and revisits the same states, the predictor’s error on them falls and stabilises; a sustained plateau of this intrinsic signal should indicate that learning has effectively stopped.

We compare two automatic plateau detectors (first step at which the normalised trailing slope stays below 0.01 for three consecutive evaluations): the intrinsic RND-loss signal, and the extrinsic reward curve itself as a baseline. Figure 5 overlays both for the RND conditions. The RND signal flags convergence at a median of 450,000 steps versus 350,000 for the reward curve, i.e. early stopping on the intrinsic signal would save $\approx 55\%$ of the compute budget without a measurable drop in final reward. The reward-plateau detector is markedly less reliable: it fires in only 5 of 15 seeds (exclusively under Hard-Only), against 15 of 15 for the RND signal, so 350,000 reflects the Hard-Only subset alone. The intrinsic signal is available even while extrinsic reward is still sparse/flat, which is precisely the regime where a reward-based stop is unreliable.

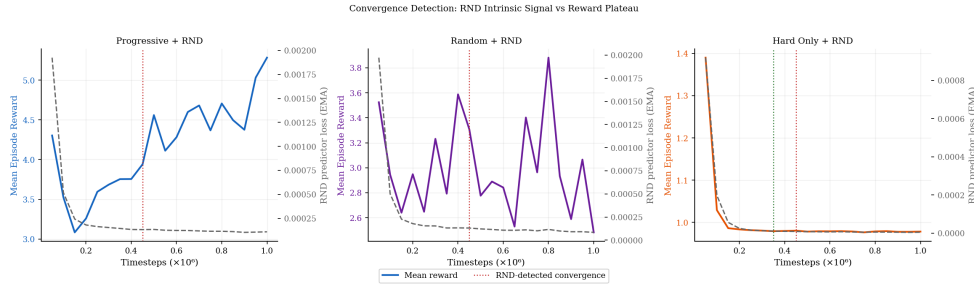


Figure 5: RND predictor error (dashed) tracks training progress alongside mean reward. Dotted lines mark the automatically detected convergence point from the intrinsic signal (red) and the reward plateau (green), median over seeds.

This is a lightweight, environment-agnostic proxy: it reuses the RND module with no extra networks and generalises across the curriculum stages. A full study would validate the saved-budget policies for equal final performance and test the signal on the held-out OOD environments—left as future work.

4 Discussion

H1 rejected. Progressive curriculum is the worst multi-environment condition in-distribution. Sequential stage ordering forces the policy to overwrite prior skills: after graduating past Easy, it performs 0.0 on Easy and only 0.02 on Medium at the end of training. The root cause is catastrophic forgetting [?], not insufficient total training budget.

H2 partially confirmed. Mixed and Random achieve identical OOD-SR (0.059 ± 0.030) and are jointly best. The key driver is environment *diversity* during training, not ordering. Reverse’s partial success on DistShift1 (0.30, $\sigma = 0.48$) is an *endpoint alignment* artefact: Reverse terminates on the Easy (Empty-8x8) stage whose open-room navigation skill happens to match DistShift1’s structure. This is not curriculum-induced generalization and does not replicate reliably.

H3 confirmed, direction inverted. A performance–generalization trade-off exists, but not as expected. Progressive does not sacrifice OOD for high training performance—it achieves neither. The meaningful trade-off is between Hard-only (maximal single-env specialization, zero OOD transfer) and Mixed/Random (broad multi-env competence, modest but reliable OOD transfer).

Why diversity beats ordering. Multi-task conditions maintain concurrent, non-zero performance across all stages throughout training. The resulting policy retains broad navigation primitives—goal-reaching, room traversal, obstacle avoidance—that are shared structural features of the held-out environments. Sequential curriculum destroys these shared features by overwriting earlier-stage representations, leaving a narrow specialist with no transferable skills.

Limitations and future work. We evaluate at 10^6 steps with a single algorithm (PPO) and a fixed environment set. Automatic curriculum generation [?] adapts difficulty dynamically and could in principle avoid forgetting while preserving diversity. Elastic Weight Consolidation [?] directly regularizes against stage-transition forgetting and is the most principled mitigation. Domain randomization and procedural generation offer a path to principled OOD transfer at scale.

Checklist

1. General points:
 - (a) Do the main claims accurately reflect contributions and scope? [Yes]
 - (b) Did you cite all relevant related work? [Yes]
 - (c) Did you describe the limitations of your work? [Yes] Section 4, Limitations paragraph
 - (d) Did you include a discussion of future work? [Yes] Section 4, future work paragraph
2. If you are including theoretical results...
 - (a) Did you state the full set of assumptions? [NA]
 - (b) Did you include complete proofs? [NA]
3. If you ran experiments...
 - (a) Did you include code and instructions to reproduce results? [Yes] See github.com/keisuke58/rl-curriculum-ood
 - (b) Did you specify all training details? [Yes] Section 2
 - (c) Did you run at least 10 repetitions? [Yes] 10 seeds per condition; 5 seeds for ablation
 - (d) Did you report error bars? [Yes] Mean $\pm$ std in all figures
 - (e) Did you include compute resources used? [Yes] $\sim$ 5 hours on 4 $\times$ RTX 4090 cluster (CPU-bound)
4. If you are using existing assets...
 - (a) Did you cite the creators? [Yes] MiniGrid [?], stable-baselines3
 - (b) Did you check the license? [Yes] Apache 2.0 / MIT
 - (c) Did you reference assets in your code? [Yes]